

Ordenamiento - Sorting

Algoritmos y Estructuras de Datos

Departamento de Computación,
Facultad de Ciencias Exactas y Naturales,
Universidad de Buenos Aires

18 de junio de 2025

- ¿Cuál fue el concepto que te pareció más importante de la clase?
- ¿Qué parte del sentís que te resultó más clara?
- ¿Qué parte creés que no entendiste bien?
- ¿Si le comentaras un “tip” a un compañero/a que faltó, cuál sería?
- Una breve devolución sobre que te pareció la modalidad de la clase.

¿Por qué sorting?

- Es uno de los problemas clásicos de computación, del área de algoritmos.

¿Por qué sorting?

- Es uno de los problemas clásicos de computación, del área de algoritmos.
- Buscar elementos en una estructura es mucho más eficiente si está ordenada.

¿Por qué sorting?

- Es uno de los problemas clásicos de computación, del área de algoritmos.
- Buscar elementos en una estructura es mucho más eficiente si está ordenada.
- Es muy común que un algoritmo ordene algo como subrutina, y así su eficiencia está atada al algoritmo de sorting que use.

¿Por qué sorting?

- Es uno de los problemas clásicos de computación, del área de algoritmos.
- Buscar elementos en una estructura es mucho más eficiente si está ordenada.
- Es muy común que un algoritmo ordene algo como subrutina, y así su eficiencia está atada al algoritmo de sorting que use.
- Tener buenos algoritmos de sorting puede ayudar a resolver muchos problemas en forma eficiente.

Un pequeño repaso

- Insertion Sort, $\Theta(n^2)$ en el peor caso.

Se va insertando en el lugar correspondiente cada elemento en un arreglo ordenado. Son n elementos e insertar ordenado cuesta $O(n)$. Trabaja in-place.

Un pequeño repaso

- Insertion Sort, $\Theta(n^2)$ en el peor caso.

Se va insertando en el lugar correspondiente cada elemento en un arreglo ordenado. Son n elementos e insertar ordenado cuesta $O(n)$. Trabaja in-place.

- Selection Sort, $\Theta(n^2)$

Busca el mínimo elemento entre una posición i y el final de la lista, lo intercambia con el elemento de la posición i e incrementa i . Buscar el menor cuesta $O(n)$ y son n elementos. Trabaja in-place.

Un pequeño repaso

- Insertion Sort, $\Theta(n^2)$ en el peor caso.

Se va insertando en el lugar correspondiente cada elemento en un arreglo ordenado. Son n elementos e insertar ordenado cuesta $O(n)$. Trabaja in-place.

- Selection Sort, $\Theta(n^2)$

Busca el mínimo elemento entre una posición i y el final de la lista, lo intercambia con el elemento de la posición i e incrementa i . Buscar el menor cuesta $O(n)$ y son n elementos. Trabaja in-place.

- MergeSort, $\Theta(n \log n)$

Algoritmo recursivo que ordena sus dos mitades y luego las fusiona. No es in-place.

Un pequeño repaso

- Insertion Sort, $\Theta(n^2)$ en el peor caso.

Se va insertando en el lugar correspondiente cada elemento en un arreglo ordenado. Son n elementos e insertar ordenado cuesta $O(n)$. Trabaja in-place.

- Selection Sort, $\Theta(n^2)$

Busca el mínimo elemento entre una posición i y el final de la lista, lo intercambia con el elemento de la posición i e incrementa i . Buscar el menor cuesta $O(n)$ y son n elementos. Trabaja in-place.

- MergeSort, $\Theta(n \log n)$

Algoritmo recursivo que ordena sus dos mitades y luego las fusiona. No es in-place.

- QuickSort, $\Theta(n^2)$ en el peor caso. $\Theta(n \log n)$ en el caso promedio.

Elige un pivote y separa los elementos entre los menores y los mayores a él. Luego, se ejecuta el mismo procedimiento sobre cada una de las partes. Es de lo mejor que hay en la práctica.

Un pequeño repaso

- Insertion Sort, $\Theta(n^2)$ en el peor caso.

Se va insertando en el lugar correspondiente cada elemento en un arreglo ordenado. Son n elementos e insertar ordenado cuesta $O(n)$. Trabaja in-place.

- Selection Sort, $\Theta(n^2)$

Busca el mínimo elemento entre una posición i y el final de la lista, lo intercambia con el elemento de la posición i e incrementa i . Buscar el menor cuesta $O(n)$ y son n elementos. Trabaja in-place.

- MergeSort, $\Theta(n \log n)$

Algoritmo recursivo que ordena sus dos mitades y luego las fusiona. No es in-place.

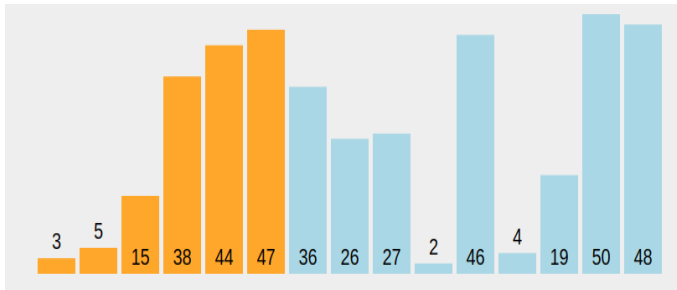
- QuickSort, $\Theta(n^2)$ en el peor caso. $\Theta(n \log n)$ en el caso promedio.

Elige un pivote y separa los elementos entre los menores y los mayores a él. Luego, se ejecuta el mismo procedimiento sobre cada una de las partes. Es de lo mejor que hay en la práctica.

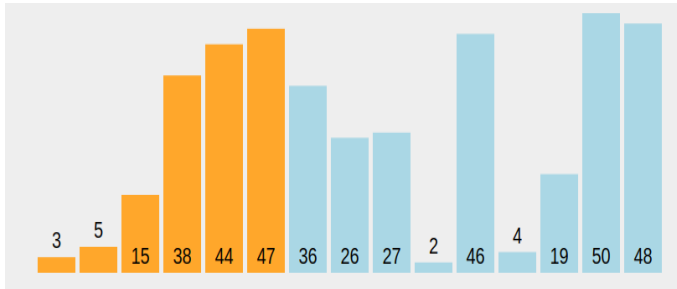
- HeapSort, $O(n + n \log n) = O(n \log n)$

Arma el Heap en $O(n)$ y va sacando los elementos ordenados pagando $O(\log n)$.

Trivia - Adivinen qué algoritmo se está usando

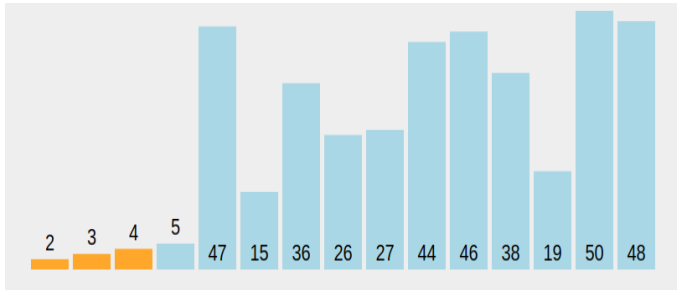


Trivia - Adivinen qué algoritmo se está usando

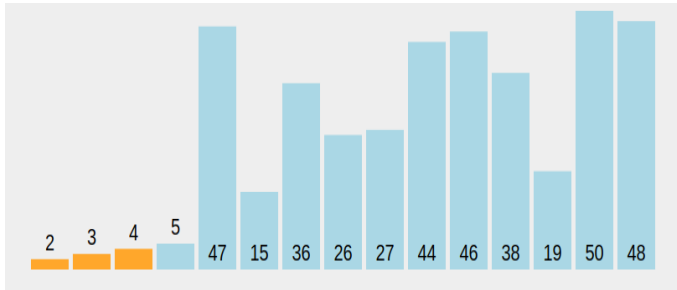


Insertion Sort

Trivia - Adivinen qué algoritmo se está usando

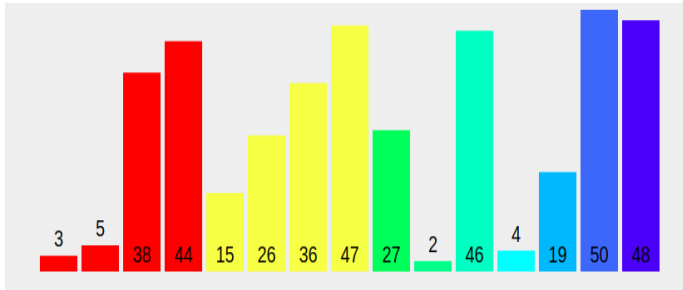


Trivia - Adivinen qué algoritmo se está usando

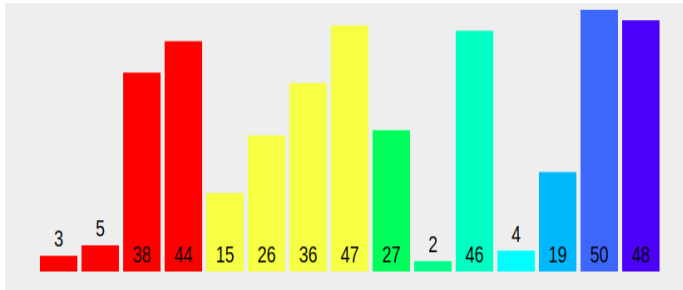


Selection Sort

Trivia - Adivinen qué algoritmo se está usando



Trivia - Adivinen qué algoritmo se está usando



Merge Sort

Actividad: Debate relámpago

Formen grupos de $4(\pm 1)$ personas, debatir las consignas que se darán.

Actividad: Debate relámpago

Formen grupos de $4(\pm 1)$ personas, debatir las consignas que se darán.

Reglas:

- Van a tener 2 minutos para sacar la mejor conclusión posible para cada consigna. Lo que sea. Todo vale.

Actividad: Debate relámpago

Formen grupos de $4(\pm 1)$ personas, debatir las consignas que se darán.

Reglas:

- Van a tener 2 minutos para sacar la mejor conclusión posible para cada consigna. Lo que sea. Todo vale.
- Serán 9 consignas, pero van a tener 11 bloques de 2m. Los dos extra son comodines para volver sobre alguna o elegir ignorar 2 veces la orden de cambio.

Actividad: Debate relámpago

Formen grupos de 4($\pm$ 1) personas, debatir las consignas que se darán.

Reglas:

- Van a tener 2 minutos para sacar la mejor conclusión posible para cada consigna. Lo que sea. Todo vale.
- Serán 9 consignas, pero van a tener 11 bloques de 2m. Los dos extra son comodines para volver sobre alguna o elegir ignorar 2 veces la orden de cambio.
- Si quieren bajar la dificultad, vale dividirse en dos equipos y abordar la mitad de las consignas cada uno. El segundo equipo empieza en la pregunta $n+1$. Vale consultarse.

Actividad: Debate relámpago

Formen grupos de 4($\pm$ 1) personas, debatir las consignas que se darán.

Reglas:

- Van a tener 2 minutos para sacar la mejor conclusión posible para cada consigna. Lo que sea. Todo vale.
- Serán 9 consignas, pero van a tener 11 bloques de 2m. Los dos extra son comodines para volver sobre alguna o elegir ignorar 2 veces la orden de cambio.
- Si quieren bajar la dificultad, vale dividirse en dos equipos y abordar la mitad de las consignas cada uno. El segundo equipo empieza en la pregunta $n+1$. Vale consultarse.
- Cada grupo comenzará por la pregunta que indique el último número de la LU (xxx**n**/xx) de la persona del grupo con el primer nombre en orden alfabético.

Actividad: Debate relámpago

Formen grupos de 4($\pm$ 1) personas, debatir las consignas que se darán.

Reglas:

- Van a tener 2 minutos para sacar la mejor conclusión posible para cada consigna. Lo que sea. Todo vale.
- Serán 9 consignas, pero van a tener 11 bloques de 2m. Los dos extra son comodines para volver sobre alguna o elegir ignorar 2 veces la orden de cambio.
- Si quieren bajar la dificultad, vale dividirse en dos equipos y abordar la mitad de las consignas cada uno. El segundo equipo empieza en la pregunta $n+1$. Vale consultarse.
- Cada grupo comenzará por la pregunta que indique el último número de la LU (xxxn/xx) de la persona del grupo con el primer nombre en orden alfabético.
- Si el número es 0, eligen cualquiera.
- Luego de la 9 va la 1.

Actividad – Debate relampago: consignas

Actividad – Debate relámpago: consignas

1. ¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

Actividad – Debate relámpago: consignas

1.¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2.¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b.¿Eso le suma utilidad al algoritmo?¿Por qué?

Actividad – Debate relámpago: consignas

1. ¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2. ¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b. ¿Eso le suma utilidad al algoritmo? ¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de que se trata? ¿es en si mismo un algoritmo útil?

Actividad – Debate relámpago: consignas

1. ¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2. ¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b. ¿Eso le suma utilidad al algoritmo? ¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de que se trata? ¿es en si mismo un algoritmo útil?

4. ¿Por qué creen que en la materia amamos MergeSort?

Actividad – Debate relámpago: consignas

1. ¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2. ¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b. ¿Eso le suma utilidad al algoritmo? ¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de qué se trata? ¿es en si mismo un algoritmo útil?

4. ¿Por qué creen que en la materia amamos MergeSort?

5. ¿Cómo funciona el algoritmo de Floyd/Heapify?

Actividad – Debate relámpago: consignas

1. ¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2. ¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b. ¿Eso le suma utilidad al algoritmo? ¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de qué se trata? ¿es en si mismo un algoritmo útil?

4. ¿Por qué creen que en la materia amamos MergeSort?

5. ¿Cómo funciona el algoritmo de Floyd/Heapify?

6. Tengo un criterio principal y uno de desempate. ¿Qué algoritmo usarían para ordenar en dos pasadas?

Actividad – Debate relámpago: consignas

1. ¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2. ¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b. ¿Eso le suma utilidad al algoritmo? ¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de qué se trata? ¿es en si mismo un algoritmo útil?

4. ¿Por qué creen que en la materia amamos MergeSort?

5. ¿Cómo funciona el algoritmo de Floyd/Heapify?

6. Tengo un criterio principal y uno de desempate. ¿Qué algoritmo usarían para ordenar en dos pasadas?

7. ¿Que criterio usarían primero en el caso anterior?

Actividad – Debate relámpago: consignas

1.¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2.¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b.¿Eso le suma utilidad al algoritmo?¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de que se trata? ¿es en si mismo un algoritmo útil?

4.¿Por qué creen que en la materia amamos MergeSort?

5,¿Cómo funciona el algoritmo de Floyd/Heapify?

6.Tengo un criterio principal y uno de desempate. ¿Qué algoritmo usarían para ordenar en dos pasadas?

7. ¿Que criterio usarían primero en el caso anterior?

8.¿Que algoritmo usarían si la entrada suele venir en orden inverso? ¿Y ordenada?

Actividad – Debate relámpago: consignas

1.¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2.¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b.¿Eso le suma utilidad al algoritmo?¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de que se trata? ¿es en si mismo un algoritmo útil?

4.¿Por qué creen que en la materia amamos MergeSort?

5,¿Cómo funciona el algoritmo de Floyd/Heapify?

6.Tengo un criterio principal y uno de desempate. ¿Qué algoritmo usarían para ordenar en dos pasadas?

7. ¿Que criterio usarían primero en el caso anterior?

8.¿Que algoritmo usarían si la entrada suele venir en orden inverso? ¿Y ordenada?

9.¿Por qué creen que les di una mecánica tan rebuscada para la actividad?